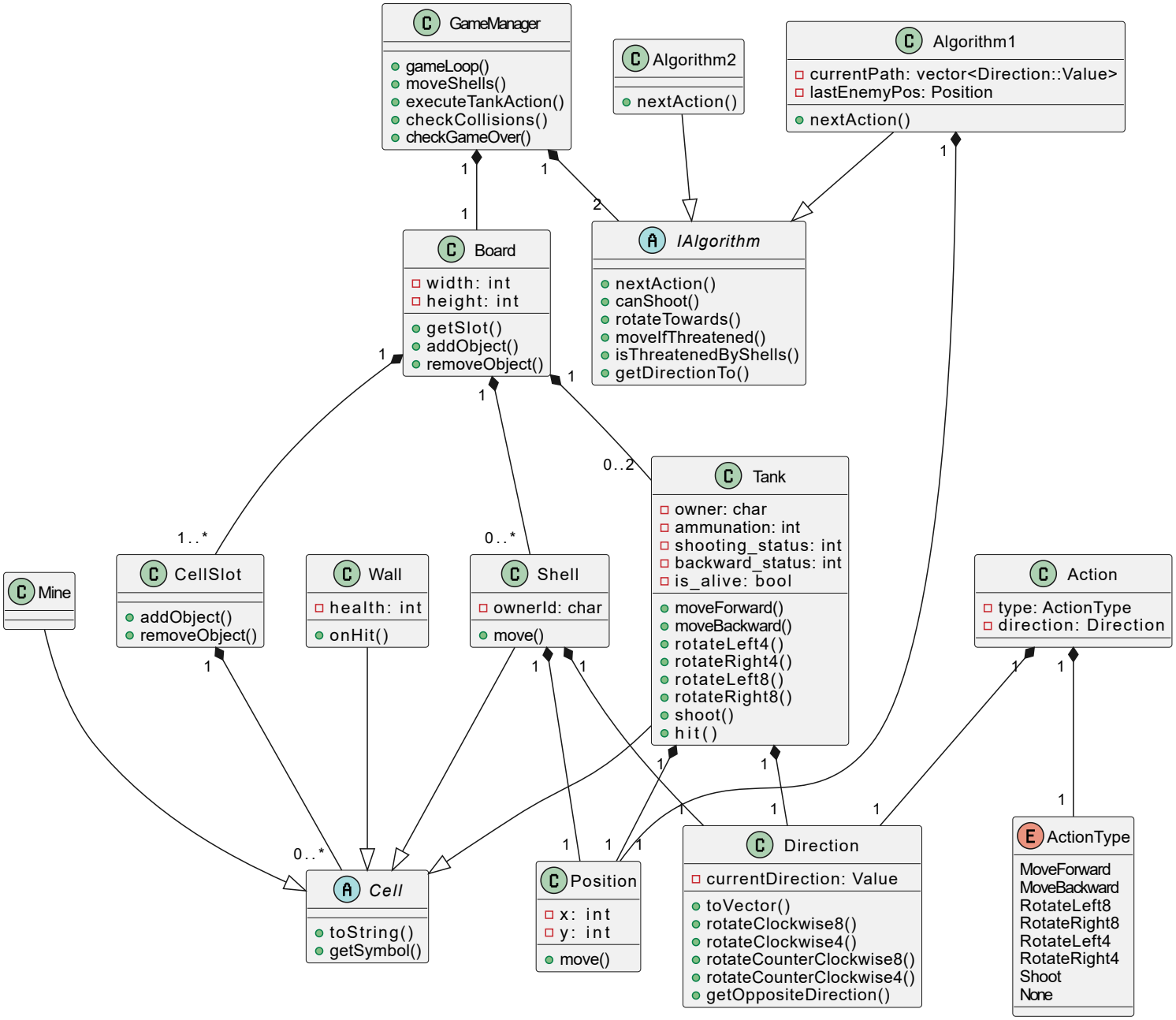
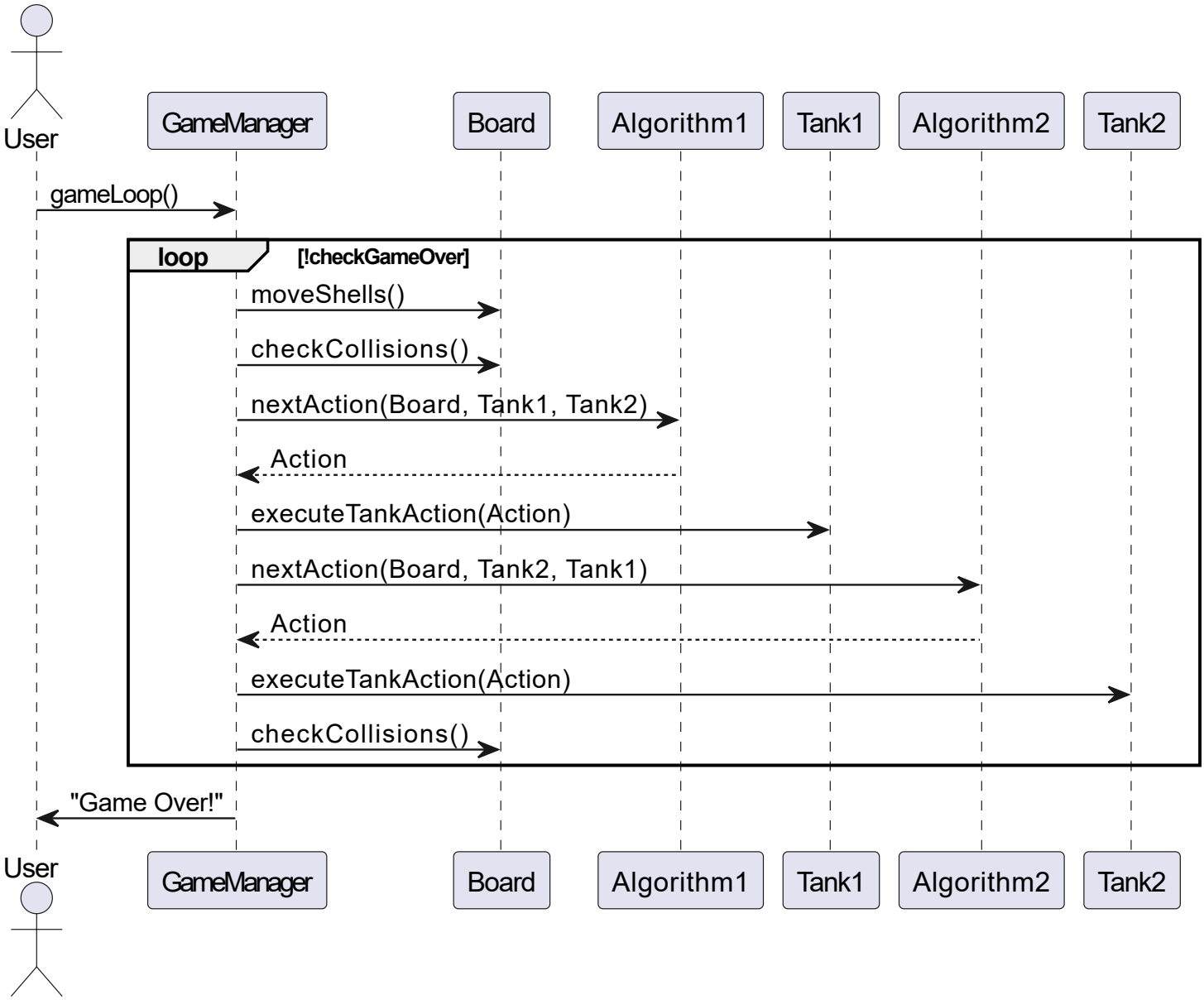

GameManager
gameLoop()
moveShells()
executeTankAction()
checkCollisions()
checkGameOver()
Algorithm2
nextAction()
Algorithm1
currentPath: vector<Direction::Value>
lastEnemyPos: Position
nextAction()
Board
width: int
height: int
getSlot()
addObject()
removeObject()
IAlgorithm
nextAction()
canShoot()
rotateTowards()
moveIfThreatened()
isThreatenedByShells()
getDirectionTo()
Tank
owner: char
ammunation: int
shooting_status: int
backward_status: int
is_alive: bool
moveForward()
moveBackward()
rotateLeft4()
rotateRight4()
rotateLeft8()
rotateRight8()
shoot()
hit()
Mine
CellSlot
addObject()
removeObject()
Wall
health: int
onHit()
Shell
ownerId: char
move()
Action
type: ActionType
direction: Direction
Cell
toString()
getSymbol()
Position
x: int
y: int
move()
Direction
currentDirection: Value
toVector()
rotateClockwise8()
rotateClockwise4()
rotateCounterClockwise8()
rotateCounterClockwise4()
getOppositeDirection()
ActionType
MoveForward
MoveBackward
RotateLeft8
RotateRight8
RotateLeft4
RotateRight4
Shoot
None


User
GameManager
Board
Algorithm1
Tank1
Algorithm2
Tank2
gameLoop()
loop
[!checkGameOver]
moveShells()
checkCollisions()
nextAction(Board, Tank1, Tank2)
Action
executeTankAction(Action)
nextAction(Board, Tank2, Tank1)
Action
executeTankAction(Action)
checkCollisions()
"Game Over!"
User
GameManager
Board
Algorithm1
Tank1
Algorithm2
Tank2

Design Considerations

Object-Oriented Design:
We used an object-oriented programming (OOP) approach to represent the various entities in the game: tanks, walls, mines, and shells.
Each entity is modeled as a class, enabling modular design and ease of maintenance.

Inheritance and Polymorphism:
Tanks, walls, mines, and shells all inherit from the abstract class Cell.
This design enables CellSlot to uniformly manage a vector of Cell* objects, regardless of their specific type.
It simplifies operations like adding, removing, or iterating over objects, and allows for easy casting when needed (for example, to check if a Cell is a Tank or a Mine).

Abstract Base Class for Algorithms:
We created an abstract class, IAlgorithm that defines shared functionality for all algorithms (e.g., chasing the enemy, shooting logic, direction handling).
This promotes code reuse and ensures that both Algorithm1 and Algorithm2 implement a consistent interface.

Composition (Aggregation):
The Board class contains (has-a) a dynamic array of CellSlot objects, and each CellSlot contains a list of Cell*.
The GameManager class contains (has-a) a Board, two IAlgorithm instances (one for each player), and two tanks.
Composition ensures clear ownership and lifecycle management of the contained objects.

Flexible Object Storage in Cells:
Each CellSlot can store multiple objects simultaneously.
This supports scenarios where, for example, a shell and a mine occupy the same cell, which was required by the game rules.

Separation of Concerns:
The design separates responsibilities clearly across classes:

- Board handles the game map and object placement.
- CellSlot manages objects inside a specific cell.
- GameManager controls the flow of the game.
- IAlgorithm and its subclasses determine the tank's next action.

Dynamic Memory Management:
Memory is managed carefully:
Every new is paired with a delete, and smart pointers (unique_ptr) are used when appropriate.
Before program termination, all dynamically created objects are deleted to avoid memory leaks.

Design Alternatives Considered

Single Object per Cell:
I considered using a single pointer per board cell instead of a vector of objects.
Rejected because:

- It would prevent supporting multiple objects (like shell + mine) in the same cell.
- It would complicate collision detection and game rule enforcement.

Using std::vector<std::vector<Cell*>> Instead of Raw Arrays:
I considered using standard vectors instead of new[] arrays for the grid.
Rejected because:

- Manual new[] allows slightly faster access and lower memory overhead.
- The board size is known at runtime, but does not dynamically change afterward.

Storing Tanks in a Vector:
Instead of having two separate pointers (tank1, tank2), I considered using a container such as std::vector<Tank*>.
Rejected for the current assignment because:

- The number of tanks is fixed (exactly 2), so a vector would add unnecessary complexity and minimal value.
- Accessing tanks individually (player 1 vs player 2) is simpler and faster with direct pointers.

However, looking ahead to future assignments or expansions of the project (where the number of tanks could vary dynamically — for example, multiple tanks entering or respawning during gameplay),
using a std::vector<Tank*> would be a more scalable and flexible design choice. This would allow easy management of an arbitrary number of tanks without requiring code restructuring.

Strategy Pattern for Algorithms:
Initially considered formalizing algorithm selection via a Strategy design pattern.
Rejected because:

- The project requires only two specific algorithms (Algorithm1 and Algorithm2), without runtime switching.

Testing Approach

Functional Testing:
I tested the program using a variety of maps and scenarios:

- Open spaces with no obstacles.
- Tanks surrounded by walls and mines.
- Shooting across long distances.
- Wrap-around movements at the edges of the board.

Edge Case Testing:
I specifically tested rare and extreme scenarios:

- Two tanks starting on the same cell.
- Attempting invalid moves (blocked by walls).
- Shooting while out of ammo.
- Handling mines and shells colliding correctly.

Logging and Debugging:
Extensive debug logs (debug_log.txt) were generated during the game execution, allowing traceability of each move, shot, and collision.

Memory Leak Testing:
I ran the program with **AddressSanitizer** enabled (a sanitizer integrated with the compiler) to detect memory leaks and invalid memory accesses.
This tool immediately reports invalid reads/writes, memory leaks, use-after-free errors, etc., during the program execution.
The tests showed no memory leaks or invalid memory behavior, confirming that all dynamically allocated memory (like dynamic Cell objects and dynamic Shell objects) is properly released at the end of the program.